Software Architecture

Hand-in Assignment

Week 11

Instructions:

- Submit your solutions through GitHub Classroom and **remember** to submit a link to your repository in BrightSpace. Otherwise, I will have a hard time distinguishing between the different groups.
- When submitting, you should push your solutions to the existing repository. In this assignment, you will primarily need to implement the missing methods in `dict.hpp`
- You **must** add comments in your programs to explain your code. Any solutions submitted without any comments will not be graded.

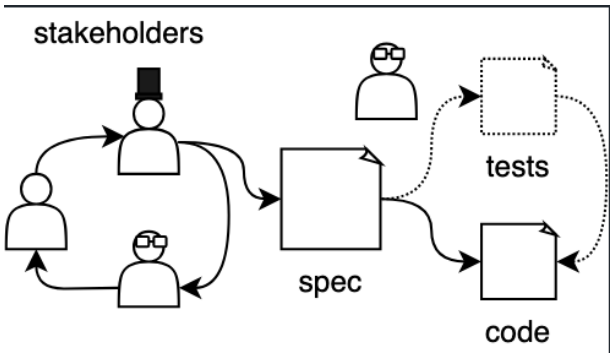
Description

In today's lecture you were introduced to the topic of software specification. During your studies the size of the projects you are working on are relatively small, and spending time on specification may seem like a waste of time. However, as the complexity of a project grows, the ability to trust other developers' code becomes absolutely essential. Experience shows that if you do not formalize how something should work, it ends up working in strange and unexpected ways.

The concepts of software specification are tightly coupled with testing. A useful analogy from modern societies is the concept of our judicial system:

- *laws* → software specification
- *police* → tests that enforce laws.

Using formal languages, laws define how people should behave, whereas the police are the ones actually verifying that people adhere to the laws. Naturally, if the laws are not enforced, i.e., the code is not tested, the specification is useless. Likewise, if the laws are too hard to understand, the police will not be able to enforce them effectively.



The grand challenge lies in how to specify software's behavior in a way that captures essential properties while remaining useful for all stakeholders involved in a project. For example, the specification should be useful to the following people:

- The developer using the requirements to guide implementation of the software.
- The tester translating the requirement specification into a set of test cases.
- The business person that is only interested in the high level features that the product provides.

There is no *'right'* way to formalize a software specification. Rather, an appropriate method should be selected based on the particular product being developed. For example ask yourself the following questions:

- Is the system safety critical, or is it okay if it breaks occasionally? Pacemaker vs. website.
- What type of application is being developed? Consider a GUI vs. a linear algebra library.

A concrete example is the Python language reference, which is the de-facto specification of the Python language:

values()

Return a new view of the dictionary's values. See the documentation of view objects.

An equality comparison between one `dict.values()` view and another will always return `False`. This also applies when comparing `dict.values()` to itself:

```
>>> d = {'a': 1}
>>> d.values() == d.values()
False
```

Another example is a more traditional requirement specification that focuses more on the application level. A snippet is shown below:

1. Introduction

[Omitted]

2. General Requirements

2.1. Description of the application

The Noise Source Location application is used to process sound intensity and sound pressure spectra. The spectra are measured at different positions around a device on which the noise sources are to be located. This is done by setting up a surface containing the measurement positions, by measuring the spectra at these positions, analyzing them to sound intensity vectors or pressure levels, and displaying the results on a picture of the surface or a fraction of the surface. The results can be displayed as a numeric map, as a vector map, or as an iso-contour plot. For plane surfaces the results can be displayed as a 3D-landscape. When the noise sources have been identified the sound power from these sources may be calculated and ranked to determine which parts of the device are of greatest interest.

2.2. Language

As English is the most common language of our potential customers, the first release of the program will be in English.

(R-1 +) **The language of this application will be in American English.**

2.3. Surface

The surface defines the exact places where the sound intensity is measured. The surface therefore has to be defined before any measurement can be performed. It is also used for presentation of the measurements. The surface is described using grids, which for manual measurements are typically built with wood and wires.

2.3.1. Grid

A grid is a plane rectangle containing m rows and n columns in a right-angled co-ordinate system. Definitions of the grids are a trade-off between accuracy of the measurement and time: Many measurement points give better accuracy but this is more time-consuming. From customers, we know that a noise location application must accept grids with a minimum of 50×50 points.

(R-2 +++) **The application must support grids of minimum 50×50 points, but ideally the grids should be boundless.**

(R-3 +++) **The application must be able to hold at least 2,500 measurement points, but ideally should be boundless.**

(R-4 +++) **It must be possible to construct a surface consisting of at least 50 grids.**

Each grid has its own co-ordinate system.

Exercise

In this exercise you will implement a `Dict` class in C++ based on a requirement specification. The class allows the insertion and retrieval of key-value pairs of different types. For example the dict class can be used to associate the name of a person with their age:

```
Dict<string, int> name_to_age;

name_to_age.set("Jane", 20);
cout << name_to_age.get("Jane").value() << endl; // prints 20
```

The requirements of the `Dict` class are formalized using the provided header `dict.hpp`:

```
/**
* @brief Associates the key with the specified value.
* If the key is already in the dictionary its value is overwritten.
*
* @param key key associated with the provided key.
* @param val value associated with the provided key.
*/
void set(K key, V val)
{
    // put code here
}

/**
* @brief Returns the number of items in the dictionary.
*
* @return the number of items in the dictionary.
*/
size_t len()
{
    return 0;
}
```

This is an example of an ambiguous specification. For example consider, (a) what is the size of the `Dict` before any element is inserted?, (b) how many elements can I store inside the dict? In cases like these, exercise your own good judgement :)

Now, your goal is to:

- Implement the `Dict` class defined in the `dict.hpp` file.
- Implement your own tests in `dict_tests.cpp`, they should
  - check that each function works as defined in the description.
  - ensure 100% coverage of the `Dict` implementation.

This is the reverse process of some of the previous exercises, where test cases were provided. Now it is your turn to convert the requirements into test cases which verifies them.

I suggest the following approach:

1. Examine the `dict.hpp` file, make sure that you understand what each function is supposed to do.
2. For each function implement a test which exercises the function.
3. Implement the method.
4. Run the tests, if they pass continue to the next function, otherwise repeat 2-3.

5. Check that the tests covers all branches of the program. Here you can use: software like `lcov` introduced in class, or if all else fails, print a unique message inside each branch of conditionals.
6. Profit

Hints

1. Key value pairs can be stored in many ways. A simple way is to store them in two separate vectors and use the same index for both the key and value of a pair.